



HASHDIT

HashDit

Smart Contract Security

ListaDao V3 LP Collateral Update

Security Audit Report

ListaDao

24 Jul 2026 – 28 Jul 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	24 Jul 2026– 28 Jul 2026
Overall Risk	Medium
Project Name	ListaDao
Github	https://github.com/lista-dao/moolah/pull/210 https://github.com/lista-dao/moolah/pull/221
Commit	feature/v3-rebalance-loss-cap

Smart Contract List

No.	Contract Name	Verdict	Details
1	V3Provider.sol	See Findings	See Findings
2	V3DexAdapter.sol	See Findings	See Findings
3	V3ProviderOracle.sol	See Findings	See Findings
4	SlisBNBV3Provider.sol	See Findings	See Findings
5	SlisBNBV3DexAdapter.sol	See Findings	See Findings
6	SlisBNBV3ProviderOracle.sol	See Findings	See Findings
7	V3Liquidator.sol	See Findings	See Findings
8	WstETHV3Provider.sol	See Findings	See Findings
9	WstETHV3DexAdapter.sol	See Findings	See Findings
10	WbETHV3Provider.sol	See Findings	See Findings
11	WbETHV3DexAdapter.sol	See Findings	See Findings
12	V3PositionLib.sol	See Findings	See Findings
13	SwapInventoryLib.sol	See Findings	See Findings
14	SmartProvider.sol	See Findings	See Findings

Findings

V3Provider

[M01] Deposit share issuance on a general TWAP pair lets passive suppliers be diluted across the fixed 5% range

Contract V3Provider.sol

Severity Medium

Description

The V3Provider.deposit() prices new shares off the position composition read at the fair price through positionAmountsAt(fairSqrtPriceX96), issuing shares = supply * frac, while withdraw() and redeemShares() settle by removing real liquidity at the live pool spot. For a rate anchored (LST) pair the fair price is the manipulation resistant exchange rate and the pool spot is arbitrage pinned to a tight 1% band, so the two prices track and the deposit is value conserving.

Description

JavaScript

```
    } else {
        // Subsequent deposit: issue shares purely proportional to the
        // existing position
        // composition - never from a pool-spot mint re-valued at fair.
        Snapshot the composition at the
        // fair price (positionAmountsAt already includes idle inventory +
        // collected fees), bind the deposit
        // to that ratio, and park the consumed tokens as IDLE rather than
        // minting into the pool. The pool
        // spot price never enters share issuance, so the mint-at-spot /
        // value-at-fair gap that let a
        // depositor be over-credited (with idle present) is eliminated.
        Value-conserving: the depositor
        // adds exactly `frac` of each composition leg and receives `frac`
        // of the supply.
        (uint256 t0, uint256 t1) =
        IV3DexAdapter(ADAPTER).positionAmountsAt(fairSqrtPriceX96);
        if (_amountsValueUsd(t0, t1) == 0) revert ZeroShares();

        uint256 frac; // WAD-scaled fraction of the existing composition
        being added
        if (t0 == 0) {
            frac = (_amount1Desired * WAD) / t1;
        } else if (t1 == 0) {
            frac = (_amount0Desired * WAD) / t0;
        } else {
            uint256 f0 = (_amount0Desired * WAD) / t0;
```

```

uint256 f1 = (_amount1Desired * WAD) / t1;
frac = f0 < f1 ? f0 : f1;
}

```

For a general, non rate anchored pair the fair price is the pool TWAP instead, which lags the real spot during volatility and can be pushed over the TWAP period. The adapter centers a non rate pair's liquidity using a fixed fallback half range of 500 ticks, roughly with a maximum of 5% above and below spot. This keeps the position two sided and actively priced across a wide spot band, so the pool spot can sit far from the TWAP while deposits are still accepted and valued.

JavaScript

```

/// @inheritdoc IV3DexAdapter
/// @dev Rate-implied (manipulation-resistant) when the subclass
supplies a non-zero LST-native
///      rate via _lstNativeRate(); otherwise falls back to the pool
TWAP.
function fairSqrtPriceX96() public view virtual returns (uint160) {
    uint256 rate = _lstNativeRate();
    if (rate == 0) return TickMath.getSqrtRatioAtTick(_twapTick());
    return _sqrtPriceX96FromRate(rate);
}

```

Impact

Because deposits are priced at the fair composition but redemptions settle at spot, every deposit taken while the two diverge shifts value equal to $\text{frac}/(1+\text{frac}) * [W(\text{spot}) - W(\text{fair})]$, where $W(P)$ is the true price value of the position's liquidity evaluated at price P . A depositor can repeatedly deposit while the gap favors them and redeems before it closes extracts that amount from passive suppliers who do not withdraw

Consider the following:

- Restrict this deposit and pricing mechanism to LST rate anchored pairs, where fair equals the manipulation resistant rate and the property already holds.
- If general TWAP pairs must be supported, gate the subsequent deposit path on a spot versus fair deviation check (reuse `_spotWithinFair()` and `maxSpotDeviationBps`) so the deposit reverts when the pool spot deviates from the fair price beyond the configured tolerance, which caps $W(\text{spot}) - W(\text{fair})$ and collapses the per deposit leak.

Recommendation

- Additionally, make the fallback range width configurable rather than fixed at 500 ticks so it can be tightened per pair, and consider adding a deposit side NAV or value neutrality guard comparable to the one already enforced on rebalance.

Status [Open/Fixed/Acknowledged]

[M02] Missing compound loss cap lets a sandwich extract value bounded by the spot deviation band

Contract V3DexAdapter.sol

Severity Medium

Description

The V3DexAdapter._collectAndCompound() routine deploys the position's idle inventory and collected fees into the Uniswap/PancakeSwap V3 pool by calling increaseLiquidity at the pool spot price with amount0Min = 0 and amount1Min = 0.

JavaScript

```
function _collectAndCompound() internal {
    if (tokenId == 0) return;

    (uint256 fees0, uint256 fees1) =
    V3PositionLib.collectAll(POSITION_MANAGER, tokenId);

    uint256 toCompound0 = fees0 + idleToken0;
    uint256 toCompound1 = fees1 + idleToken1;
    if (toCompound0 == 0 && toCompound1 == 0) return;

    // Never add liquidity at a manipulated pool spot. This compound
    runs (permissionlessly) as a
    // side effect of deposit/withdraw/redeemShares with amountMin = 0,
    so a flash-loan that skews spot
    // could make the vault add liquidity at a bad price and get
    sandwiched on the back-swap. fair is
    // rate-anchored, so if spot deviates from it we SKIP the re-add
    (not revert - must not brick the
    // user's hot-path op) and hold the collected fees as idle (valued
    at fair, unexposed) until a later
    // compound when spot ~= fair.
    if (!_spotWithinFair()) {
        idleToken0 = toCompound0;
        idleToken1 = toCompound1;
        emit CompoundSkippedSpotDeviated(spotSqrtPriceX96(),
        fairSqrtPriceX96());
        return;
    }

    // One-sided inventory relative to the active range yields zero
    addable liquidity, which would
```

Description

```

        // revert inside pool.mint (require(amount > 0)). Hold everything as
        idle and skip this round
        // instead of reverting, so deposit / withdraw / redeemShares (which
        reach here permissionlessly)
        // are never bricked by a single-sided balance. The idle is deployed
        on a later compound once the
        // opposite leg arrives or a rebalance recenters the range.
        uint128 addable = LiquidityAmounts.getLiquidityForAmounts(
            spotSqrtPriceX96(),
            TickMath.getSqrtRatioAtTick(tickLower),
            TickMath.getSqrtRatioAtTick(tickUpper),
            toCompound0,
            toCompound1
        );
        if (addable == 0) {
            idleToken0 = toCompound0;
            idleToken1 = toCompound1;
            emit CompoundSkippedNoLiquidity(toCompound0, toCompound1);
            return;
        }

        (uint128 liquidityAdded, uint256 used0, uint256 used1) =
        V3PositionLib.increaseLiquidity(
            POSITION_MANAGER,
            TOKEN0,
            TOKEN1,
            tokenId,
            toCompound0,
            toCompound1,
            0,
            0
        );

        idleToken0 = toCompound0 - used0;
        idleToken1 = toCompound1 - used1;

        // Emit the amounts actually consumed by increaseLiquidity, not the
        raw inputs - the leftover
        // (toCompound - used) is held as idle, so the raw figures would
        overstate what was compounded.
        emit Compounded(used0, used1, liquidityAdded);
    }

```

The only protection against adding liquidity at a manipulated price is the `_spotWithinFair()` gate, which permits the compound to proceed as long as the spot price is within `maxSpotDeviationBps` of the fair price (default 100 bps, i.e. 1%). Because the deployment is a mint with no per-token minimum and this logic is reachable permissionlessly through `V3Provider.compound()` (and also through the deposit, withdraw, and redeem paths), an attacker can atomically wrap a front-run swap, the

compound call, and a back-run swap around the mint. The front-run pushes spot to the edge of the allowed band, the vault adds liquidity at that skewed ratio, and the back-run restores spot and captures the difference.

Impact

The value extracted per compound is bounded by roughly the deviation band multiplied by the size of the lump being deployed (idle inventory plus uncollected fees). There is currently no cap on how much value a single compound may lose and no limit on how much idle is deployed per call, so a fixed 1% band does not bound the absolute loss on a large position.

Additionally, note that the affected amounts is potentially higher considering the new creditIdle introduced means new depositor underlying assets is not immediately added within the V3DexAdapter held NPM position.

Recommendation

Consider also introducing an admin adjustable per compound loss BPS and maxDailyLossUsd guard for compounding within the permissionless compound function.

This allows the admin to monitor large compounding actions and potentially prevent stealing of pending deposits. This can also allow flexibility of switching off losses when a large loss is already compounded to accommodate large volumes of deposit..

Status

[Open/Fixed/Acknowledged]

[L01] Idle funds in V3DexAdapter bear impermanent loss the depositor did not choose

Contract V3DexAdapter.sol

Severity Low

Description

Description

In V3DexAdapter, idle funds are counted in NAV at face value, but they are eventually deployed into the pool at the spot price prevailing at that later time via the permissionless compound. This new 2-step deposit to liquidity addition design is introduced to remove the mint-at-spot/value-at-fair over-credit and is value-conserving.

Impact

When the idle amount enters the pool, it realizes whatever impermanent loss and spot conditions exist at that moment, and this outcome is shared across all holders rather than borne only by the depositor who contributed the idle funds. As a result, a depositor can be exposed to impermanent loss they did not actively choose, and the difference between the face valuation and the eventual spot deployment is socialized. This behavior is bounded and consistent with normal LP mechanics, so the impact is low.

Recommendation	Document this valuation and deployment behavior clearly so depositors understand that idle funds valued at face will later enter the pool at spot and absorb the IL conditions at deployment time. Consider valuing idle funds in a way that better reflects their eventual spot deployment, or deploying idle funds promptly via a keeper to reduce the window in which spot conditions can diverge from the face valuation, so the impermanent loss is not unexpectedly socialized among all holders.
Status	[Open/Fixed/Acknowledged]

[I01] Default maxDailyLossUsd of \$1000 can self-DoS legitimate rebalances in V3Provider

Contract	V3Provider.sol
Severity	Informational
Description	Description The V3Provider._guardedRebalance() function enforces a daily loss cap as a fixed USD ceiling that does not account for position size. <pre> JavaScript if (navAfter < navBefore) { uint256 loss = navBefore - navAfter; uint256 today = block.timestamp / 1 days; if (today != dailyLossDay) { dailyLossDay = today; dailyLossAccum = 0; } uint256 accum = dailyLossAccum + loss; if (accum > maxDailyLossUsd) revert DailyLossExceeded(); dailyLossAccum = accum; emit RebalanceDailyLossAccrued(today, loss, accum); } </pre>
Impact	Impact The default value of maxDailyLossUsd = \$1000 applies uniformly regardless of how large the underlying vault is. For a large vault, a legitimate recenter triggered by a genuine rate move can realize combined slippage and impermanent loss of > \$1000, which causes the transaction to revert. Because rebalance is the only path to recenter the position, this leaves the position stranded out of range until the next UTC day. While stranded, the position earns no fees and is valued by the rate anchored oracle rather than by where the liquidity actually sits, producing an inaccurate valuation. The same limitation applies over the course of a volatile day. A series of small but legitimate rebalances can accumulate and exhaust the daily cap, after which a

	genuinely needed recenter later in the same day is blocked until the cap resets at the next UTC day.
Recommendation	Consider performing active monitoring of loss accumulation from rebalancing and increasing maxDailyLossUsd via setMaxDailyLossUsd when required.
Status	[Open/Fixed/Acknowledged]

[I02] Missing zero value validation in setMaxRebalanceLossBp and setMaxDailyLossUsd can block rebalances

Contract	V3Provider.sol
Severity	Informational
Description	Description The V3Provider.setMaxRebalanceLossBp() and V3Provider.setMaxDailyLossUsd() functions do not validate that the supplied value is non zero before storing it. <pre> JavaScript /// @notice Set the per-rebalance fair-NAV loss cap, in ppm (LOSS_DENOM = 1e6). onlyRole MANAGER. function setMaxRebalanceLossBp(uint256 _maxRebalanceLossBp) external onlyRole(MANAGER) { if (_maxRebalanceLossBp > LOSS_DENOM) revert InvalidParam(); maxRebalanceLossBp = _maxRebalanceLossBp; emit MaxRebalanceLossBpChanged(_maxRebalanceLossBp); } /// @notice Set the per-UTC-day cumulative rebalance loss cap, 8-decimal USD. onlyRole MANAGER. function setMaxDailyLossUsd(uint256 _maxDailyLossUsd) external onlyRole(MANAGER) { maxDailyLossUsd = _maxDailyLossUsd; emit MaxDailyLossUsdChanged(_maxDailyLossUsd); } </pre> Impact If either limit is accidentally set to zero, the rebalance loss guards become impossible to satisfy, since any realized slippage or loss would exceed a zero ceiling. This causes legitimate rebalance operations to revert, leaving positions unable to recenter until an administrator corrects the misconfiguration.
Recommendation	Consider adding input validation to both setter functions that rejects a zero value and reverts with a clear error.
Status	[Open/Fixed/Acknowledged]

[I03] Rebalance inventory swap skips the per swap loss guard for non rate pairs where preservation of the guard relies on convention

Contract V3Provider.sol & V3DexAdapter.sol

Severity Informational

Description

The rebalance conversion in V3DexAdapter._convertToOptimalRatio() bounds the swap's execution loss through _requireSwapLossWithinCap(), which anchors the received amount to the LST to native exchange rate so a compromised BOT cannot realize more than maxSwapLossBp on the converted leg. Note that this guard returns early whenever rate == 0, which is the case for a pure TWAP pair (a pair whose subclass does not override _lstNativeRate() and therefore has no rate reference).

```
JavaScript
function _requireSwapLossWithinCap(
    bool sellToken0,
    uint256 total0,
    uint256 total1,
    uint256 newTotal0,
    uint256 newTotal1,
    uint256 rate
) internal view {
    if (rate == 0) return;
```

Description

Impact

Nothing in _convertToOptimalRatio() prevents a non rate pair from supplying non empty swapData, so such a pair can execute a swap with the rate anchored per swap cap fully skipped. In that path the only remaining bounds on the swap are the internal spent <= amountIn and received >= amountOutMin checks in SwapInventoryLib (where amountOutMin is supplied by the BOT), the swapPairWhitelist set by the manager, and the coarser rebalance level guards in V3Provider._guardedRebalance() (the per rebalance fair NAV cap maxRebalanceLossBp, default 2%, and the per day cap maxDailyLossUsd, default 1000 USD).

There is a comment instructing that a subclass overriding the hook must preserve an equivalent guard refers to _convertToOptimalRatio(), which is declared virtual and therefore can be overridden. The guard helper _requireSwapLossWithinCap() is not virtual and cannot be overridden, though it is internal so an overriding hook can still call it. Preservation of the per swap guard across a hook override is therefore enforced only by convention, not by the compiler, and even when the helper is called it no ops for a non rate pair because of the rate == 0 early return.

Recommendation Consider making the documented convention enforceable so a future non rate pair cannot run an unguarded swap.

Status	[Open/Fixed/Acknowledged]
--------	---------------------------

[I04] Missing V3Liquidation event in liquidate function of V3Liquidator

Contract	V3Liquidator.sol
----------	------------------

Severity	Informational
----------	---------------

Description	Description The V3Liquidator.liquidate() function does not emit a V3Liquidation event when it executes. This is inconsistent with the flashLiquidate() function and with other liquidator contracts, which do emit appropriate liquidation event.
	<pre>JavaScript function liquidate(bytes32 id, address borrower, uint256 seizedAssets, uint256 repaidShares) external nonReentrant onlyRole(BOT) { require(marketWhitelist[id], NotWhitelisted()); IMoolah.MarketParams memory params = IMoolah(MOOLAH).idToMarketParams(id); // Pre-approve Moolah to pull the repayment; cleared after the call. params.loanToken.safeApprove(MOOLAH, type(uint256).max); IMoolah(MOOLAH).liquidate(params, borrower, seizedAssets, repaidShares, ""); params.loanToken.safeApprove(MOOLAH, 0); }</pre> Impact Without the event, liquidations performed through liquidate() are not recorded on chain in a way that off chain systems, monitoring tools, and indexers can track. This reduces transparency and makes it harder to audit and reconcile liquidation activity.

Recommendation	Consider emitting a V3Liquidation event inside the liquidate() function with the relevant liquidation parameters, matching the event emission already present in flashLiquidate()
----------------	---

Status	[Open/Fixed/Acknowledged]
--------	---------------------------

[I05] Redeemed leg amounts use persistent storage for a value that is only needed within the transaction

Contract	V3Liquidator.sol
----------	------------------

Severity	Informational
----------	---------------

Description	Description
-------------	-------------

The V3Liquidator tracks the redeemed leg amounts through two persistent storage variables, `_redeemedAmount0` and `_redeemedAmount1`, whose only role is to pass the amounts produced inside the `onMoolahLiquidate` callback back up to `flashLiquidate` so they can be reported in the V3Liquidation event. No fund movement or profitability check depends on them; profitability is judged separately from the loan token balance delta.

JavaScript

```
/// @dev Pass-back of the leg amounts redeemed inside
onMoolahLiquidate, so flashLiquidate can report
///     the actual redeemed amount0 / amount1 in its V3Liquidation
event rather than 0. Appended at
///     the end of the storage layout (upgrade-safe). flashLiquidate
zeroes them before each liquidate
///     so a non-redeeming flash liquidation - or a stale value from
an earlier call - reports 0.
uint256 private _redeemedAmount0;
uint256 private _redeemedAmount1;
```

Impact

Because they live in persistent storage, each liquidation pays the extra cost of writing to storage, and the values remain in storage after the transaction ends. They are zeroed only at the start of the next `flashLiquidate`, so between transactions the contract keeps the last liquidation's amounts in state for no functional reason. A value whose lifetime is a single transaction is a natural fit for transient storage (EIP-1153), which is cheaper and is automatically discarded when the transaction ends, although this is a gas and cleanliness improvement, not a correctness or security issue.

Consider declaring both variables using the native transient data location

Retain the explicit zeroing of both variables at the start of `flashLiquidate`, given transient storage persists across separate calls within the same transaction and is cleared only at transaction end, so the start of call reset is still required to stop a first flash liquidation's amounts from leaking into a second flash liquidation performed in the same atomic transaction, in particular on the non redeeming path where the callback does not overwrite them.

Recommendation

Since V3Liquidator is an upgradeable (UUPS) contract, account for the storage layout change, given transient variables do not occupy persistent storage slots, so moving `_redeemedAmount0` and `_redeemedAmount1` out of persistent storage frees the two trailing slots they currently use. They are the last declared state variables, so nothing after them shifts, but reserve those slots or document them as retired so a future upgrade does not misalign the layout.

Status

[Open/Fixed/Acknowledged]

[I06] Stray native forwarded on a non wrapped native pair is left untracked in V3DexAdapter

Contract SwapInventoryLib.sol

Severity Informational

Description

The SwapInventoryLib.swap() function only wraps native coin delivered during a rebalance conversion back into the wrapped native ERC20 when either the input or output token of the swap is the wrapped native token. This is the correct behavior for every currently deployed pair, since SlisBNBV3DexAdapter (slisBNB/WBNB), WstETHV3DexAdapter (wstETH/WETH), and WbETHV3DexAdapter (wbETH/WETH) always include the wrapped native as one leg, so any native received is wrapped and booked into the totals.

Description

JavaScript

```
// Wrap any native this call delivered - a native-out venue's
proceeds (instantWithdraw → BNB) or a
// native-in venue's unspent refund - back into the wrapped-native
ERC-20, so it is booked into the
// totals via the spent/received deltas and never stranded. Only a
wrapped-native leg can legitimately
// deliver native; wrap the delta ONLY when either leg is the
wrapped-native token. If NEITHER leg is
// (an ERC-20-ERC-20 swap on a future non-native pair), any native
that appears is an unsolicited
// donation the venue forwarded (e.g. a router that sweeps its
native balance; an attacker can seed it
// with 1 wei) - it is not part of this swap's accounting, so leave
it untouched rather than reverting
// and bricking the rebalance.
if ((tokenIn == wrappedNative || tokenOut == wrappedNative) &&
address(this).balance > beforeNative) {
    IWBNB(wrappedNative).deposit{ value: address(this).balance -
beforeNative }();
}
```

Impact

For a hypothetical future pair where neither leg is the wrapped native token, a swap venue that forwards stray native (for example a router that sweeps its own native balance, or an attacker who seeds the venue with 1 wei) will cause that native to land on the adapter without being wrapped. This surplus native is not included into idleToken0 or idleToken1, and no valuation view reads address(this).balance, so it sits entirely outside the adapter's accounting.

The impact is limited to trapped surplus, not lost user principal. If a venue attempted to pay the swap output as native instead of the expected output ERC20, the measured received amount would fall below amountOutMin and the swap would revert, so user inventory can never be silently converted into stranded native. The untracked native also does not affect collateral valuation, health checks, oracle pricing, or liquidations, and the change removes a previous denial of service where the same dust would revert the rebalance. V3DexAdapter and its subclasses currently expose no sweep or rescue function, so the only recovery path is a UUPS upgrade that adds one.

Consider the following:

- Add a rescueNative() function restricted to the admin role that transfers the resting native balance, since native is never part of these adapters' accounting.
- Add a rescueToken() function that only allows withdrawing the balance above the accounted idle amount (balanceOf(address(this)) minus idleToken0 or idleToken1 for the tracked tokens), preserving the invariant that accounted idle never exceeds the adapter's balance.

Recommendation

Status [Open/Fixed/Acknowledged]